

Rapport Technique - System Architect Lab

Linux, Ordonnancement CPU et Conteneurisation Docker

LIU 2026 - Introduction aux Systemes d'Exploitation

Equipe / Team: Ahmed Cheikh 12530219 and Varha Mohamed Mahmoud 12530130

dépôt GitHub: <https://github.com/Spilled-mcoffee/System-Architect-Lab>

- Automatisation Shell avec Bash
- Simulation d'ordonnancement CPU: Round Robin et SRTF
- Infrastructure Docker: Debian personnalisée, MySQL, Apache et CRUD automatisé

Introduction

Ce rapport presente la realisation du mini-projet System Architect Lab. Le projet combine trois dimensions importantes des systemes d'exploitation modernes: l'administration Linux par script Bash, la modelisation de politiques d'ordonnancement CPU et le deploiement d'une infrastructure conteneurisee avec Docker.

L'approche adoptee consiste a organiser chaque composant dans un dossier separe, a automatiser les operations repetitives et a tester les elements techniques dans un environnement Docker reproductible. Les scripts Bash et Python sont executes dans un conteneur Debian personnalise afin d'eviter toute modification risquee sur la machine hote.

Matrice de contribution

La repartition suivante presente les responsabilites principales de chaque membre. Les deux membres ont participe aux tests, a la verification et a la preparation finale du livrable.

Membre	Responsabilites principales	Contribution
Ahmed Cheikh	Automatisation Bash, configuration Docker Compose, integration MySQL/Apache, tests de deploiement, documentation.	50%
Varha Mohamed Mahmoud	Simulation d'ordonnancement CPU, verification des resultats, automatisation CRUD, validation du rapport et demonstration.	50%

1. Automatisation Shell avec Bash

1.1 Objectif

La premiere partie du projet consiste a developper un script d'administration nomme admin_systeme.sh. Ce script automatise la creation d'un compte utilisateur, la creation d'un espace de travail personnel, l'affectation du proprietaire et la configuration des permissions d'accès.

1.2 Fonctionnement du script

- Verification des privileges administrateur avant l'execution.
- Lecture du nom d'utilisateur depuis le terminal.
- Verification de l'existence du compte afin d'eviter les doublons.
- Creation de l'utilisateur avec son repertoire personnel.
- Creation d'un dossier workspace dans le repertoire de l'utilisateur.
- Application de chown pour affecter le proprietaire et chmod 700 pour limiter l'accès au proprietaire.
- Simulation de l'affectation d'un quota disque afin d'illustrer le principe sans modifier la configuration avancee du systeme de fichiers.

1.3 Execution dans le conteneur Debian

Pour rendre le test plus sur et plus reproductible, le script Bash n'est pas execute directement sur la machine hote. Le dossier local bash/ est monte dans le conteneur Debian sous le chemin /lab/bash. Ainsi, la creation d'utilisateurs et de dossiers se produit uniquement dans l'environnement isole du conteneur.

```
docker exec -it debian_lab bash
ls -la /lab/bash
bash /lab/bash/admin_systeme.sh
id testuser
ls -ld /home/testuser/workspace
```

Le resultat attendu est la creation d'un utilisateur de test, l'apparition d'un repertoire /home/testuser/workspace et des permissions de type drwx----- correspondant au mode 700.

2. Modelisation de l'ordonnancement CPU

2.1 Objectif

La deuxième partie consiste à comparer deux algorithmes d'ordonnancement: Round Robin et Shortest Remaining Time First (SRTF). La simulation calcule les temps d'attente, les temps de retour et produit une représentation de type diagramme de Gantt.

2.2 Données de simulation

Processus	Temps d'arrivée	Temps d'exécution
P1	0	7
P2	2	4
P3	4	1
P4	5	4

2.3 Round Robin

Round Robin attribue à chaque processus un quantum de temps fixe. Dans cette implémentation, le quantum choisi est de 2 unités. Si un processus n'est pas terminé après son quantum, il est remplacé à la fin de la file d'attente. Cet algorithme favorise l'équité, mais peut augmenter le temps d'attente moyen à cause des changements fréquents de contexte.

2.4 SRTF

SRTF choisit à chaque instant le processus dont le temps restant est le plus court. Il est préemptif et peut interrompre un processus lorsqu'un nouveau processus plus court arrive. Cet algorithme tend à réduire le temps d'attente moyen, mais il peut pénaliser les processus longs.

2.5 Exécution dans Docker

Le script Python `scheduling.py` est également exécuté dans le conteneur Debian. Le dossier `scheduling/` est monté sous `/lab/scheduling`. Comme l'image Debian personnalisée contient déjà Python, aucune installation manuelle n'est nécessaire au moment de la démonstration.

```
docker exec -it debian_lab bash
python3 --version
python3 /lab/scheduling/scheduling.py
```

3. Infrastructure et conteneurisation Docker

3.1 Architecture générale

La troisième partie de l'infrastructure repose sur Docker Compose. L'architecture contient trois services principaux: un conteneur Debian personnalisé, un serveur MySQL et un serveur HTTP Apache.

Service	Rôle	Configuration principale
debian_lab	Environnement Linux de test	Image personnalisée construite avec <code>Dockerfile.debian</code>
mysql_server	Base de données étudiants	MySQL 8.0, port hôte 3307 vers port conteneur 3306, volume persistant
apache_server	Serveur HTTP	Image <code>httpd</code> , port hôte 8080 vers port conteneur 80, page web personnalisée

3.2 Image Debian personnalisée

Une image Debian personnalisée a été créée afin d'éviter l'installation répétitive de Python et des outils d'administration à chaque recreation du conteneur. Le fichier Dockerfile.debian part de debian:stable-slim et installe python3, passwd, procs, iproute2, iputils-ping et nano.

```
FROM debian:stable-slim

RUN apt-get update && apt-get install -y --no-install-recommends \
    python3 passwd procs iproute2 iputils-ping nano \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /lab
CMD ["bash"]
```

Dans docker-compose.yml, le service debian_lab utilise build afin de construire cette image locale. Les dossiers bash/ et scheduling/ sont montés dans le conteneur pour permettre l'exécution directe des scripts du projet.

```
debian_lab:
  build:
    context: .
    dockerfile: Dockerfile.debian
  image: system-architect-debian:1.0
  container_name: debian_lab
  tty: true
  stdin_open: true
  volumes:
    - ../bash:/lab/bash
    - ../scheduling:/lab/scheduling
```

3.3 MySQL, CRUD et volume persistant

Le service MySQL initialise une base de données nommée étudiants. Le port interne 3306 est exposé sur le port 3307 de la machine hôte afin d'éviter un conflit avec un serveur MySQL local déjà présent. Les opérations CRUD sont automatisées par un script Bash qui transmet le fichier sql/crud.sql au conteneur MySQL.

```
./scripts/run_crud.sh
```

Le volume mysql_data est monté sur /var/lib/mysql. Ce chemin correspond à l'emplacement interne dans lequel MySQL stocke ses fichiers de données. Grâce à ce volume, les données peuvent persister même si le conteneur est arrêté ou recréé. La commande docker compose down conserve le volume, tandis que docker compose down -v le supprime.

3.4 Serveur Apache et page personnalisée

Le service Apache utilise l'image officielle httpd. Le port 80 du conteneur est exposé sur le port 8080 de la machine hôte. Le dossier local web/ est monté sur /usr/local/apache2/htdocs afin que le serveur affiche une page HTML personnalisée au lieu de la page par défaut.

```
apache_server:
  image: httpd:latest
  container_name: apache_server
  ports:
    - "8080:80"
  volumes:
    - ../web:/usr/local/apache2/htdocs
```

La vérification s'effectue dans le navigateur à l'adresse <http://localhost:8080>.

3.5 Protocole de test

1. Construire et démarrer l'infrastructure: `docker compose up -d --build`.
2. Vérifier les conteneurs avec `docker compose ps`.
3. Exécuter le script Bash dans le conteneur Debian.
4. Exécuter le simulateur Python dans le conteneur Debian.
5. Afficher la page Apache dans le navigateur sur <http://localhost:8080>.
6. Exécuter les opérations CRUD avec `./scripts/run_crud.sh`.

7. Verifier que MySQL, Apache et Debian restent operationnels.

Conclusion

Le projet realise une integration coherente entre administration Linux, simulation d'ordonnancement et conteneurisation. L'ajout d'une image Debian personnalisee ameliore la reproductibilite, car les outils necessaires sont integres directement dans l'image. L'execution des scripts Bash et Python dans le conteneur renforce la securite du test et montre l'interet de Docker comme environnement controle. Enfin, l'association de MySQL, d'un volume persistant, d'un serveur Apache et de scripts d'automatisation fournit une infrastructure claire, testable et presentable.